

Detalhamento Técnico

Telas desenvolvidas

Foram desenvolvidas quatro telas:

- **products** — tela principal, com o catálogo de produtos disponíveis e a opção de realizar a compra.
- **invoice** — exibida após a confirmação de uma compra. Mostra a visualização da nota fiscal gerada e permite a impressão.
- **product-create** — formulário de cadastro de novos produtos.
- **invoice-list** — listagem de todas as notas fiscais emitidas. Notas ainda não impressas (status OPEN) aparecem com a opção de impressão disponível.

Funcionalidades implementadas

- Cadastro de novos produtos
- Compra de produtos, com emissão da nota fiscal
- Listagem de notas fiscais
- Layout responsivo
- Mensagens de erro para o usuário
- Tela de processamento durante a realização da compra

Detalhamento técnico

- Ciclos de vida do Angular;

O principal ciclo de vida utilizado foi o `ngOnInit`, responsável pela inicialização dos componentes e pelo carregamento dos dados por meio dos serviços. Não houve necessidade de utilizar outros lifecycles para a implementação do projeto.

- Uso da biblioteca RxJS;

RxJS foi utilizado para tratar as requisições HTTP de forma assíncrona, com `Observable` e `subscribe()`. Também foram utilizados os operadores `map`, para tratar os dados retornados pela API, e `delay`, para simular um tempo de processamento durante a emissão da nota fiscal.

- Bibliotecas foram utilizadas e para qual finalidade;
 - Angular Router — navegação entre as telas da aplicação.
 - Angular HttpClient — comunicação com os microserviços.
 - Angular Forms — construção e validação dos formulários.
 - pgx — conexão do backend com o PostgreSQL.

- Zerolog — geração de logs estruturados no backend.
- CORS — permite a comunicação entre o frontend e os microsserviços.
- Como foi realizado o gerenciamento de dependências no Golang;

O gerenciamento foi feito através do Go Modules, com os arquivos `go.mod` e `go.sum`. O `go.mod` define as dependências utilizadas pelo projeto, enquanto o `go.sum` armazena as versões e os hashes de verificação de cada uma.

- Frameworks do Golang;

Foi utilizado o framework Gin para a criação das APIs HTTP dos microsserviços, responsável pela definição das rotas, pelo recebimento das requisições e pelo envio das respostas.

- Erros e exceções no backend;

O tratamento de erros no backend foi feito em duas camadas:

- Handlers: tratamento de erros esperados, retornando o código HTTP correspondente a cada situação — 400 para dados inválidos, 404 para recursos não encontrados, 409 para conflitos (como estoque insuficiente) e 500 para erros internos.
- Middleware de Recovery: captura exceções inesperadas (`panic`), registra o erro através do Zerolog e retorna uma resposta adequada ao cliente, evitando que o servidor seja derrubado.

Arquitetura de microsserviços

O sistema é dividido em dois microsserviços independentes, que se comunicam por HTTP:

- **Serviço de Estoque (Stock Service)** — responsável pelo cadastro de produtos, consulta de produtos (geral e individual) e baixa de estoque, com validação de saldo insuficiente.
- **Serviço de Faturamento (Billing Service)** — responsável pela criação e listagem de notas fiscais, controle de status, numeração sequencial, impressão e comunicação com o Serviço de Estoque.

Os dois serviços rodam em containers separados, orquestrados por Docker Compose e configurados por variáveis de ambiente. O PostgreSQL não está incluído no Compose: roda separadamente, no ambiente local, e é acessado pelos dois serviços por conexão de rede.

A numeração das notas fiscais é gerada pelo próprio PostgreSQL, por meio de uma sequence (`invoice_number_seq`) consultada com `nextval()`. Essa escolha eliminou a geração aleatória de número que existia anteriormente no Angular, além de impedir que o número seja definido pelo cliente.

Persistência em banco de dados

O banco utilizado é o PostgreSQL, no banco `nfs_issuer`, com as tabelas `products`, `invoices` e `invoice_items`. Todos os cadastros são persistidos fisicamente: produtos, notas fiscais e os itens de cada nota.

Tratamento de falha de microsserviço

Foi implementado um cenário de falha entre os dois serviços: quando o Billing Service não consegue se comunicar com o Stock Service, o erro é identificado, registrado nos logs e retornado por meio de uma resposta HTTP apropriada. O Angular exibe essa mensagem ao usuário, com a opção de tentar novamente. Ao restabelecer o Stock Service, o sistema volta a funcionar normalmente, sem necessidade de reiniciar os demais componentes.

Itens opcionais

- **Tratamento de concorrência** — a baixa de estoque é realizada de forma atômica no PostgreSQL, com o gerenciamento de conexões feito pelo `pgxpool`. Uma proteção transacional completa contra duas impressões simultâneas da mesma nota fiscal não foi implementada.
- **Uso de inteligência artificial** — não implementado.
- **Idempotência** — não implementada.

Como rodar

No terminal faça:

```
git clone https://github.com/EduardoStockler1/Korp-Teste-Eduardo-Stockler
```

E então:

```
cd Korp-Test-Eduardo-Stockler
```

Para rodar o frontend:

```
npm start
```

Para rodar o backend:

```
docker compose up
```

Um detalhe importante: o Postgres não está incluído no Docker Compose. Ele está rodando separadamente, localmente. Isso significa que, pra rodar o projeto do zero, além do docker-compose up:

Ter o Postgres instalado e rodando localmente (ou em um container a parte se preferir). Criar o banco `nfs_issuer` com as tabelas `products`, `invoices` e `invoice_items` (e a sequence `invoice_number_seq`).

Configurar as variáveis de ambiente dos microsserviços com o host/porta/usuário/senha certos pra esse PostgreSQL — senão o backend sobe pelo Docker, mas não consegue conectar no banco.